

ROTABUS – SISTEMA DE CONSULTA DE PASSAGENS E HORÁRIOS

André Henrique de Oliveira, Diego Pereira da Silva, Edclevison da Silva Souza Almeida, Eduardo Maia, Gabrielle Marques Silva, Graziele dos Santos Rodrigues, Isaias Gomes Dally Junior, Lorena Franciele Dias da Silva, Luan Henrique Borges Silva, Pedro Pilla Soares, Rafael Sampaio Barbosa, Thiago Ianuch Marin

Curso de Ciência da Computação
Ânima Educação – Modalidade Live
Brasil
e-mail: rotabus.project@gmail.com

Resumo: O projeto RotaBus tem como objetivo solucionar a dificuldade de acesso às informações do transporte rodoviário de passageiros no Brasil, especialmente em rotas intermunicipais e viagens para o interior, onde a ausência de canais digitais dificulta a consulta de horários, destinos e valores de passagens. A proposta consiste no desenvolvimento de um sistema distribuído capaz de centralizar informações de viagens rodoviárias em nível nacional, utilizando Java Spring Boot no backend e uma API REST para integração entre diferentes interfaces e usuários do sistema. A aplicação permitirá a consulta de destinos por origem, visualização de horários de partida e chegada, consulta de preços das passagens e gerenciamento de viagens por empresas de transporte. Este artigo apresenta os conceitos de sistemas distribuídos aplicados ao desenvolvimento da solução, destacando sua arquitetura, funcionalidades e benefícios para a modernização do acesso às informações do transporte rodoviário.

Palavras-chave: Sistemas distribuídos, transporte rodoviário, API REST, Spring Boot, mobilidade digital.

Abstract: *The RotaBus project aims to solve the difficulty of accessing passenger road transport information in Brazil, especially on intercity routes and trips to rural areas, where the lack of digital channels makes it difficult to consult schedules, destinations, and ticket prices. The proposal consists of developing a distributed system capable of centralizing road travel information nationwide, using Java Spring Boot in the backend and a REST API for integration between different interfaces and system users. The application will allow destination searches by origin, visualization of departure and arrival times, ticket price consultation, and trip management by transport companies. This article presents the concepts of distributed systems applied to the development of the solution, highlighting its architecture, functionalities, and benefits for modernizing access to road transport information.*

Keywords: *Distributed systems, road transport, REST API, Spring Boot, digital mobility.*

Introdução

O transporte rodoviário de passageiros representa um dos principais meios de deslocamento no Brasil, especialmente em viagens interestaduais e intermunicipais realizadas por populações que dependem do acesso a cidades do interior e regiões afastadas dos grandes centros urbanos. Segundo dados da Agência Nacional de Transportes Terrestres (ANTT) [1], o setor movimenta milhões de passageiros anualmente e envolve milhares de empresas habilitadas para operação em todo o território nacional. Entretanto, apesar da relevância desse modal para a mobilidade da população, ainda existem dificuldades relacionadas ao acesso rápido e centralizado às informações de viagens rodoviárias.

A maioria das empresas de transporte concentra informações de horários, rotas, destinos e valores de passagens em canais descentralizados, obrigando passageiros a recorrerem a atendimentos presenciais, ligações telefônicas ou consultas individuais em diferentes plataformas. Esse cenário é ainda mais crítico em rotas destinadas ao interior, onde frequentemente há ausência de canais digitais organizados e atualizados. Além de impactar negativamente a experiência dos usuários, a falta de integração gera sobrecarga nos guichês e centrais de atendimento, reduzindo a eficiência operacional das empresas e aumentando o tempo de espera para obtenção de informações básicas. O próprio contexto regulatório do setor exige que empresas mantenham comunicação clara e atualizada com os passageiros em situações de atraso ou alteração de viagens, reforçando a importância de soluções digitais centralizadas.

Nesse cenário, observa-se a necessidade de sistemas capazes de integrar e disponibilizar informações de transporte rodoviário de forma acessível, organizada e escalável. Os conceitos de sistemas distribuídos tornam-se fundamentais, uma vez que permitem a comunicação simultânea entre múltiplos clientes, serviços e bases de dados por meio de arquiteturas cliente-servidor e APIs RESTful. A utilização de tecnologias modernas possibilita maior disponibilidade das informações,

integração entre diferentes interfaces e melhor desempenho no atendimento das demandas dos usuários.

Assim, este trabalho propõe, desenvolve e analisa o sistema RotaBus, uma aplicação distribuída voltada para consulta de informações do transporte rodoviário de passageiros em nível nacional. A solução é implementada utilizando Java Spring Boot no backend e comunicação via API REST, permitindo consultas de destinos, horários, rotas e valores de passagens, além do gerenciamento de viagens por empresas de transporte. O projeto busca aplicar conceitos de sistemas distribuídos na construção de uma plataforma capaz de modernizar e otimizar o acesso às informações do setor rodoviário brasileiro, contribuindo para maior eficiência operacional e autonomia dos usuários.

Desenvolvimento

O desenvolvimento do sistema RotaBus fundamenta-se nos princípios da computação distribuída, buscando integrar informações de transporte rodoviário provenientes de diferentes empresas operantes em diversas regiões do Brasil. A proposta foi concebida para solucionar a fragmentação das informações atualmente existentes no setor, permitindo que usuários consultem destinos, horários, preços e disponibilidade de viagens em uma única plataforma. Dessa forma, o sistema busca reduzir a dependência de atendimentos presenciais e consultas em múltiplos canais, promovendo maior acessibilidade às informações de mobilidade rodoviária.

Segundo Tanenbaum e Van Steen [2], um sistema distribuído pode ser definido como um conjunto de computadores independentes que se apresenta aos usuários como um sistema único e coerente. Essa definição está diretamente relacionada à proposta do RotaBus, uma vez que o sistema pretende reunir informações provenientes de diferentes empresas de transporte e disponibilizá-las de forma transparente aos usuários finais. Nesse contexto, o passageiro não precisa conhecer a origem dos dados nem a tecnologia utilizada por cada empresa participante, percebendo apenas uma única plataforma de consulta.

A arquitetura proposta segue o modelo cliente-servidor distribuído. Nesse modelo, os usuários interagem com o sistema por meio de aplicações clientes (web ou mobile), e as requisições são encaminhadas para o RotaBus, que atua como uma camada de integração e roteamento (Middleware). O banco local armazena dados estruturais e atua como cache operacional, enquanto as informações dinâmicas de viagens podem ser sincronizadas ou consultadas a partir das APIs das empresas parceiras.

A comunicação ocorre por meio de uma API REST, implementada em Java Spring Boot. Essa abordagem está alinhada aos princípios arquiteturais definidos por

Fielding [3]. A comunicação baseada em protocolos padronizados favorece a interoperabilidade, permitindo que as empresas terceiras mantenham o acesso aos seus dados de forma estruturada, enquanto o RotaBus atua como o nó centralizador que orquestra as requisições distribuídas.

A estrutura interna da aplicação foi concebida em camadas, seguindo boas práticas de engenharia de software destacadas por Fowler [4], separando as responsabilidades de apresentação, regras de negócio e persistência de dados. O banco de dados desempenha um papel fundamental na centralização e estruturação das informações, servindo como base para o cache operacional e relatórios. Para abstrair a complexidade do domínio rodoviário em um modelo relacional, a arquitetura utiliza o mapeamento objeto-relacional (ORM) provido pela especificação JPA (Java Persistence API). A Listagem 1 apresenta a entidade central do sistema, Viagem, demonstrando como atributos essenciais — como origem, destino, horários e relacionamento com a empresa operadora — são consolidados para garantir consistência nas consultas.

Listagem 1: Mapeamento objeto-relacional da entidade Viagem utilizando JPA.

```
Java
package br.com.rotabus.model;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
public class Viagem {

    @Id
    @GeneratedValue(strategy=
GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(nullable = false)
    private Cidade cidadeOrigem;

    @ManyToOne
    @JoinColumn(nullable = false)
    private Cidade cidadeDestino;

    @Column(nullable = false)
    private LocalDateTime horarioSaida;

    @Column(nullable = false)
    private                      LocalDateTime
horarioChegada;

    @Column(nullable = false, precision
= 10, scale = 2)
    private BigDecimal valorPassagem;

    @Column(nullable = false, precision
= 10, scale = 2)
```

```

private BigDecimal distanciaKm;

@Column(nullable = false)
private LocalDateTime criadoEm;

@ManyToOne
@JoinColumn(name = "empresa_id",
nullable = false)
private Empresa empresa;
}

```

Um dos aspectos mais relevantes em sistemas distribuídos modernos é a garantia de escalabilidade e disponibilidade, conforme discutido por Kleppmann [5]. Como as consultas à API REST podem ocorrer simultaneamente por milhares de usuários, o sistema não deve retornar cargas massivas de dados que esgotem a memória do servidor central. Para mitigar esse risco de gargalo de rede, a camada de persistência do RotaBus implementa nativamente o suporte à paginação. A Listagem 2 demonstra o repositório de viagens utilizando a interface Pageable do Spring Data JPA, que garante a transferência de dados em lotes (chunks) otimizados.

Listagem 2: Repositório de dados com controle de escalabilidade por paginação e filtros dinâmicos.

```

Java
package br.com.rotabus.repository;

import br.com.rotabus.model.Empresa;
import br.com.rotabus.model.Viagem;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import java.time.LocalDateTime;

public interface ViagemRepository
extends JpaRepository<Viagem, Long> {

    Page<Viagem>
findByCidadeOrigemNomeIgnoreCaseAndCidade
DestinoNomeIgnoreCaseAndHorarioSaidaBetwe
en(
        String origem,
        String destino,
        LocalDateTime inicioBusca,
        LocalDateTime fimDia,
        Pageable pageable
    );

    Page<Viagem> findByEmpresa(Empresa
empresa, Pageable pageable);
}

```

Além da eficiência na recuperação de dados, sistemas que congregam múltiplas organizações exigem isolamento lógico de informações, um conceito conhecido como *Multi-tenancy* (Multilocação). É imperativo assegurar que o acesso de uma operadora de

transporte aos nós administrativos não exponha os dados sensíveis de uma operadora concorrente. Segundo Sommerville [6], os mecanismos de controle de acesso são componentes vitais em arquiteturas compartilhadas.

Para solucionar este requisito, a camada de serviços do RotaBus foi integrada ao contexto de segurança da aplicação. A Listagem 3 ilustra o controle operacional realizado pela classe ViagemService. Ao processar uma requisição de listagem, o sistema identifica o originador no contexto de segurança e aplica o particionamento lógico: usuários com privilégios administrativos têm visibilidade global, enquanto usuários corporativos recebem apenas a página de dados atrelada à sua própria empresa.

Listagem 3: Serviço de negócios aplicando isolamento de dados (Multi-tenancy) e segurança de contexto.

```

Java
package br.com.rotabus.service;

import br.com.rotabus.model.Usuario;
import br.com.rotabus.model.Viagem;
import br.com.rotabus.repository.ViagemRepository;
import org.springframework.stereotype.Service;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;

@Service
public class ViagemService {

    private final ViagemRepository
viagemRepository;
    private final CustomUserDetailsService
usuarioService;

    public
ViagemService(ViagemRepository
viagemRepository, CustomUserDetailsService
usuarioService) {
        this.viagemRepository =
viagemRepository;
        this.usuarioService =
usuarioService;
    }

    public Page<Viagem>
listarViagensEmpresa(Pageable pageable) {
        Usuario usuario =
usuarioService.usuarioLogado();

        if
(usuarioService.usuarioLogadoEhAdmin()) {
            return
viagemRepository.findAll(pageable);
        }
    }
}

```

```
        return  
viagemRepository.findByEmpresa(usuario.ge  
tEmpresa(), pageable);  
    }  
}
```

Por fim, o RotaBus representa uma aplicação prática dos principais conceitos estudados na disciplina de Sistemas Distribuídos. A solução unifica o mapeamento objeto-relacional eficiente, a gestão de escalabilidade via paginação de APIs REST e a segurança rigorosa de dados em ambientes compartilhados. O uso da pilha tecnológica Java Spring Boot prova ser altamente adequado para centralizar e digitalizar o complexo ecossistema do transporte rodoviário nacional, otimizando os processos mercadológicos e promovendo um acesso à informação democrático e eficiente para os passageiros.

Resultados

A implementação da arquitetura distribuída do RotaBus resultou em um sistema funcional, modular e aderente aos requisitos de escalabilidade e segurança em ambientes compartilhados. O backend, desenvolvido em Java Spring Boot, estruturou a API RESTful de forma estável, permitindo a comunicação eficiente e padronizada com diferentes interfaces clientes, sejam guichês físicos ou plataformas móveis de consulta.

A validação do código confirmou que a abstração da entidade Viagem, gerenciada via JPA/Hibernate, atende com sucesso à complexidade das rotas intermunicipais, mantendo a integridade referencial com as entidades de cidades e empresas operadoras. Testes estruturais na API comprovaram a eficácia das estratégias de gestão de recursos: a aplicação de paginação (Pageable) nas consultas evitou o consumo excessivo de memória do servidor, permitindo que buscas abrangentes por datas e destinos fossem fatiadas e entregues de maneira otimizada ao cliente final, impedindo sobrecargas no canal de comunicação.

No aspecto da segurança distribuída, os resultados evidenciam o correto funcionamento do isolamento de inquilinos (*Multi-tenancy*). A verificação de contexto inserida na camada de serviço garantiu que, apesar de habitarem o mesmo ecossistema central, as empresas parceiras tenham visão estritamente particionada de suas próprias rotas, enquanto os administradores retêm o controle global do sistema. Isso confere robustez à plataforma para atrair e integrar múltiplas operadoras de transporte sem o risco de vazamento de dados estratégicos ou interferência cruzada entre nós organizacionais.

Discussão

A estruturação do projeto demonstra que a aplicação de arquiteturas integradoras baseadas em APIs REST é uma resposta tecnologicamente viável e eficiente para o

problema da fragmentação de informações no transporte rodoviário brasileiro. Conforme apresentado na introdução deste trabalho, a dependência de guichês físicos e canais descentralizados penaliza especialmente os usuários de rotas do interior. O RotaBus cumpre o papel de mascarar essa divisão mercadológica sob uma única interface, atingindo a transparência de acesso preconizada por Tanenbaum e Van Steen [2].

A escolha do Spring Boot provou ser um diferencial para a coesão do projeto. A capacidade de lidar nativamente com a paginação de dados e a segurança de contexto simplifica a orquestração do sistema, reduzindo a complexidade de manutenção. Diferente de plataformas isoladas tradicionais, a API padronizada do RotaBus permite que o sistema cresça e acomode novos parceiros sem a necessidade de reestruturação de suas bases legadas.

Entretanto, centralizar as operações em um único repositório conceitual levanta questões inerentes a gargalos de performance futuros quando houver alta concorrência de acessos simultâneos. Embora a paginação trate o fluxo de leitura de maneira excelente no estágio atual do protótipo, uma adoção em larga escala em períodos de alta demanda exigiria estratégias arquiteturais adicionais. Para trabalhos futuros, sugere-se a avaliação do padrão CQRS (*Command Query Responsibility Segregation*) para separar as bases ou fluxos de leitura e escrita, garantindo que as operações de atualização e gerenciamento de cadastros pelas empresas não concorrem por recursos computacionais com as milhares de consultas de pesquisa realizadas em tempo real pelos passageiros.

Conclusão

O desenvolvimento do projeto RotaBus permitiu ilustrar como conceitos de sistemas distribuídos são aplicados na resolução de problemas reais. A centralização do transporte rodoviário em uma única plataforma amplia a acessibilidade dos usuários aos cronogramas de viagens, reduzindo a fragmentação e tornando a consulta mais eficiente.

A implementação da aplicação utilizando Java Spring Boot e arquitetura baseada em serviços evidenciou a importância da comunicação entre componentes, da organização em camadas e da disponibilização de recursos por meio de uma API padronizada. Essas características contribuíram para a construção de uma solução estruturada, capaz de suportar futuras integrações e expansões, mantendo a organização e a escalabilidade do sistema.

Diante dos resultados obtidos, conclui-se que o projeto atingiu os objetivos propostos ao disponibilizar uma plataforma para consulta e gerenciamento de informações do transporte rodoviário. Além de atender

aos requisitos definidos, o desenvolvimento do RotaBus possibilitou a aplicação prática dos conceitos estudados na disciplina, demonstrando o potencial dos sistemas distribuídos para modernizar serviços e facilitar o acesso à informação. Como perspectivas futuras, destacam-se a integração com bases de dados de empresas de transporte, a ampliação dos serviços oferecidos e a implementação de funcionalidades que permitam atualização e consulta de informações em tempo real.

Agradecimentos

Os autores agradecem à Ânima Educação pelos recursos acadêmicos disponibilizados, que contribuíram para a realização deste trabalho. Agradecem também aos professores da disciplina de Sistemas Distribuídos e Mobile pelos conhecimentos transmitidos, orientações e materiais de apoio fornecidos ao longo do semestre, fundamentais para a compreensão dos conceitos abordados e para o desenvolvimento do projeto RotaBus.

Referências

- [1] Agência Nacional de Transportes Terrestres (ANTT). Transporte Rodoviário Interestadual e Internacional de Passageiros no Brasil. Brasília: ANTT, 2025. Disponível em: <https://www.antt.gov.br>. Acesso em: 19 maio 2026.
- [2] TANENBAUM, Andrew S.; STEEN, Maarten Van. Sistemas Distribuídos: Princípios e Paradigmas. 2. ed. São Paulo: Pearson Prentice Hall, 2007.
- [3] FIELDING, Roy T. Architectural Styles and the Design of Network-based Software Architectures. Tese (Doutorado) – University of California, Irvine, 2000.
- [4] FOWLER, Martin. Padrões de Arquitetura de Aplicações Corporativas. Porto Alegre: Bookman, 2006.
- [5] KLEPPMANN, Martin. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. Sebastopol: O'Reilly Media, 2017.
- [6] SOMMERVILLE, Ian. Engenharia de Software. 9. ed. São Paulo: Pearson Prentice Hall, 2011.